

TDD en Java — Exercices Jour 2

Doubles de Test, Mockito & Techniques d'Écriture

Sommaire

- Rappel & pré-requis
- Niveau 1 — Découverte des doubles
 - Ex 1.1 — Identifier le bon double
 - Ex 1.2 — Stub manuel
 - Ex 1.3 — Spy manuel
 - Ex 1.4 — Fake Repository
- Niveau 2 — Mockito fondamentaux
 - Ex 2.1 — Premier mock Mockito
 - Ex 2.2 — Stubbing conditionnel
 - Ex 2.3 — Vérification des interactions
 - Ex 2.4 — ArgumentCaptor
- Niveau 3 — Mockito avancé

Rappel & pré-requis

Avant de commencer, vérifiez que votre projet contient la dépendance Mockito :

```
<dependency>  
  <groupId>org.mockito</groupId>  
  <artifactId>mockito-junit-jupiter</artifactId>  
  <version>5.5.0</version>  
  <scope>test</scope>  
</dependency>
```

Et l'import statique utile dans vos classes de test :

```
import static org.mockito.Mockito.*;  
import static org.mockito.ArgumentMatchers.*;  
import static org.junit.jupiter.api.Assertions.*;  
import org.mockito.junit.jupiter.MockitoExtension;  
import org.junit.jupiter.api.extension.ExtendWith;
```

Niveau 1 — Découverte des doubles

Objectif : Comprendre les différents types de doubles de test et savoir les implémenter sans bibliothèque.

Durée estimée : 1h30

Ex 1.1 — Identifier le bon double

Contexte : Pour chacune des situations ci-dessous, identifiez le type de double le plus adapté (Dummy / Stub / Fake / Spy / Mock) et justifiez votre choix.

Situation A

```
// Vous testez un service de calcul de prix.  
// Le service prend un Logger en paramètre mais vous ne voulez  
// pas vérifier ce qui est loggué, juste que le calcul fonctionne.  
public class PrixService {  
    public PrixService(Logger logger) { ... }  
    public double calculer(double ht, double taux) { ... }  
}
```

→ Type de double pour `Logger` : _____

→ Justification : _____

Situation B

Ex 1.2 — Stub manuel

Contexte : Implémentez manuellement un Stub pour le service météo, **sans utiliser Mockito.**

Interface à stuber :

```
public interface MeteoService {  
    String getMeteo(String ville);        // "Ensoleillé", "Nuageux", etc.  
    int getTemperature(String ville);     // en degrés Celsius  
    boolean isRisquePluie(String ville); // true si risque de pluie  
}
```

Service à tester :

```
public class ConseillerVetements {  
    private final MeteoService meteoService;  
  
    public ConseillerVetements(MeteoService meteoService) {  
        this.meteoService = meteoService;  
    }  
}
```

Ex 1.3 — Spy manuel

Contexte : Implémentez un Spy pour capturer les appels à un service de notification.

Interface à espionner :

```
public interface NotificationService {  
    void notifierParEmail(String destinataire, String message);  
    void notifierParSMS(String telephone, String message);  
    void notifierParPush(String userId, String titre, String corps);  
}
```

Étape 1 — Créez le Spy

```
class SpyNotificationService implements NotificationService {  
    // Enregistrez TOUS les appels :  
    // - Nombre d'appels à chaque méthode  
    // - Arguments de chaque appel (liste de destinataires, messages)  
  
    // Méthodes d'inspection à créer :  
    // - getNombreEmailsEnvoyes()
```

Ex 1.4 — Fake Repository

Contexte : Implémentez un `InMemoryProductRepository` (Fake) qui se comporte comme un vrai repository mais en mémoire.

Interface :

```
public interface ProductRepository {  
    Product save(Product product);           // sauvegarde et retourne avec ID généré  
    Optional<Product> findById(Long id);  
    Optional<Product> findByReference(String ref);  
    List<Product> findByCategorie(String categorie);  
    List<Product> findAll();  
    void deleteById(Long id);  
    boolean existsByReference(String ref);  
    long count();  
}
```

Contraintes pour le Fake :

- Les IDs sont **auto-incrémentés** (commence à 1)

Niveau 2 — Mockito fondamentaux

Objectif : Maîtriser la création de mocks, le stubbing et les vérifications de base avec Mockito.

Durée estimée : 1h45

Ex 2.1 — Premier mock Mockito

Contexte : Réécrivez les tests de l'Ex 1.2 et 1.3 en utilisant Mockito au lieu des stubs/spy manuels.

Exercice A — Réécriture du ConseillerVetements

```
@ExtendWith(MockitoExtension.class)
class ConseillerVetementsTest {

    @Mock
    MeteoService meteoService;    // ← Mockito crée le stub

    @InjectMocks
    ConseillerVetements conseiller;

    @Test
    void conseiller_GrandFroid_RecommandeManteauChaud() {
        // Configurer le comportement du mock
        when(meteoService.getTemperature(anyString())).thenReturn(5);
        when(meteoService.isRisquePluie(anyString())).thenReturn(false);

        String conseil = conseiller.conseiller("Paris");

        assertEquals("Manteau chaud obligatoire", conseil);
    }
}
```

Ex 2.2 — Stubbing conditionnel

Contexte : Le service ci-dessous appelle une API de paiement. Configurez des stubs conditionnels.

```
public interface PaiementAPI {
    PaiementResult payer(String carteId, double montant);
    boolean rembourser(String transactionId);
    StatutCarte getStatutCarte(String carteId);
}

public enum StatutCarte { VALIDE, EXPIREE, BLOQUEE }

public class PaiementService {
    private final PaiementAPI api;

    public Commande payer(Commande commande, String carteId) {
        StatutCarte statut = api.getStatutCarte(carteId);
        if (statut == StatutCarte.EXPIREE)
            throw new CarteExpireException(carteId);
        if (statut == StatutCarte.BLOQUEE)
            throw new CarteBloqueException(carteId);

        PaiementResult result = api.payer(carteId, commande.getMontant());
        if (!result.isSuccess())
            throw new PaiementRefuseException(result.getRaison());

        commande.setStatut(StatutCommande.PAYEE);
        commande.setTransactionId(result.getTransactionId());
    }
}
```

Ex 2.3 — Vérification des interactions

Contexte : Vérifiez les interactions avec les mocks dans les scénarios suivants.

Scénario — CommandeService

```
public class CommandeService {  
    private final StockService stockService;  
    private final PaiementService paiementService;  
    private final EmailService emailService;  
    private final CommandeRepository repository;  
  
    public Commande passerCommande(Panier panier, String carteId) {  
        // 1. Vérifier le stock pour chaque article  
        // 2. Réserver le stock  
        // 3. Payer  
        // 4. Sauvegarder la commande  
        // 5. Envoyer email de confirmation  
        return commande;  
    }  
}
```

Ex 2.4 — ArgumentCaptor

Contexte : Utilisez ArgumentCaptor pour inspecter les arguments passés aux mocks.

Service :

```
public class InscriptionService {
    private final UserRepository repository;
    private final EmailService emailService;
    private final AuditService auditService;

    public User inscrire(InscriptionRequest request) {
        User user = new User(
            request.getNom(),
            request.getEmail().toLowerCase().trim(),
            genererMotDePasseHash(request.getMotDePasse()),
            Role.USER,
            LocalDateTime.now()
        );

        User userSauvegarde = repository.save(user);

        emailService.envoyer(
            user.getEmail(),
            "Bienvenue " + user.getNom() + " !",
            "Votre compte a été créé avec succès."
        );

        auditService.enregistrer(
            "INSCRIPTION",
            user.getId(),
            Map.of("email", user.getEmail(), "ip", request.getIpOrigine())
        );
    }
}
```

Niveau 3 — Mockito avancé

Objectif : Maîtriser les fonctionnalités avancées de Mockito : InOrder, Spy, comportements dynamiques, organisation des fixtures.

Durée estimée : 2h00

Ex 3.1 — Vérification d'ordre

Contexte : L'ordre des appels est critique pour un processus de validation de commande.

Processus à tester :

PROCESSUS DE VALIDATION (ordre OBLIGATOIRE) :

1. Authentifier l'utilisateur
2. Valider la commande (métier)
3. Vérifier le stock
4. Calculer le prix final (avec remises)
5. Débiter le paiement
6. Mettre à jour le stock
7. Générer la facture
8. Envoyer la confirmation

Interfaces :

```
public interface AuthService      { User authentifier(String token); }  
public interface ValidationService { void valider(Commande c); }
```

Ex 3.2 — @Spy partiel

Contexte : Utilisez `@Spy` pour tester partiellement une classe en stubant seulement certaines méthodes.

Classe à espionner :

```
public class RapportService {
    private final DataSource dataSource;

    // Méthode publique à tester
    public Rapport genererRapportMensuel(int mois, int annee) {
        List<Vente> ventes = chargerVentes(mois, annee); // appel BDD
        double chiffreAffaires = calculerCA(ventes);      // logique pure
        double marge = calculerMarge(ventes);             // logique pure
        String resume = genererResume(chiffreAffaires, marge); // logique pure
        return new Rapport(mois, annee, chiffreAffaires, marge, resume);
    }

    // Méthode protégée → stubbable via @Spy
    protected List<Vente> chargerVentes(int mois, int annee) {
        // Accès BDD → impossible à tester directement
        return dataSource.query("SELECT ...");
    }
}
```


Ex 3.3 — Comportements dynamiques

Contexte : Utilisez `thenAnswer` pour simuler des comportements complexes.

Scénario A — Simuler une BDD auto-incrémentée

```
// Le repository doit assigner un ID auto-généré à la sauvegarde
when(repository.save(any(User.class))).thenAnswer(invocation -> {
    User user = invocation.getArgument(0);
    // TODO : retourner un nouvel objet User avec un ID généré
    // L'ID doit s'incrémenter à chaque appel (1, 2, 3, ...)
});
```

Scénario B — Simuler une latence réseau

```
// API qui répond lentement (pour tester les timeouts)
when(apiExterne.appeler(anyString())).thenAnswer(invocation -> {
    // TODO : simuler un délai de 100ms
    return new ApiResponse("OK");
});
```

Ex 3.4 — Fixtures et Builders

Contexte : Créez des fixtures réutilisables pour les tests de l'ensemble du jour 2.

Étape 1 — Créez un Builder pour User

```
public class UserBuilder {
    // Valeurs par défaut sensibles
    private Long id = null;
    private String nom = "Utilisateur Test";
    private String email = "test@exemple.com";
    private String motDePasseHash = "hash$sécurisé";
    private Role role = Role.USER;
    private boolean actif = true;
    private LocalDateTime dateCreation = LocalDateTime.now();

    // Méthodes de configuration chaînées
    public UserBuilder avecId(Long id) { ... }
    public UserBuilder avecNom(String nom) { ... }
    public UserBuilder avecEmail(String email) { ... }
    public UserBuilder commAdmin() { ... }
    public UserBuilder inactif() { ... }
    public UserBuilder creeLe(LocalDateTime date) { ... }

    public User build() { ... }
```

Niveau 4 — Projet intégré

Objectif : Implémenter un système complet avec des doubles de test appropriés, Mockito avancé et fixtures de qualité.

Durée estimée : 3h00

Ex 4.1 — Système de réservation complet

Contexte : Implémentez en TDD un système de réservation de billets d'événements.

Domaine métier :

RÈGLES DE RÉSERVATION :

- Un utilisateur peut réserver 1 à 5 billets par événement
- Le paiement est débité immédiatement à la réservation
- Un email de confirmation est envoyé après paiement réussi
- En cas d'échec paiement : aucun billet réservé, aucun email
- L'annulation est possible jusqu'à 48h avant l'événement
- L'annulation entraîne un remboursement et un email
- Une annulation tardive (< 48h) : remboursement à 50%
- Chaque réservation a un code unique de 8 caractères alphanumériques

Interfaces :

```
public interface EvenementRepository {  
    Optional<Evenement> findById(Long id);  
    void reserverPlaces(Long evenementId, int nbPlaces);  
    void libererPlaces(Long evenementId, int nbPlaces);  
}
```

Corrigés partiels

Corrigé — Ex 1.1 (réponses)

Situation	Type de double	Justification
Logger	Dummy	Requis par le constructeur mais non utilisé dans ce test
EmailService	Mock	On veut vérifier <i>que</i> l'email est envoyé à la bonne adresse
MeteoAPI	Stub	On contrôle les données retournées sans vérifier les appels
ProduitRepository	Fake	On veut une implémentation fonctionnelle complète mais légère

Corrigé — Ex 2.1 (amorce)

```
@ExtendWith(MockitoExtension.class)
class ConseillerVetementsTest {

    @Mock
    MeteoService meteoService;

    @InjectMocks
    ConseillerVetements conseiller;

    @Test
    void conseiller_TempMoinsde10_RecommandeManteauChaud() {
        // Arrange
        when(meteoService.getTemperature("Paris")).thenReturn(5);
        when(meteoService.isRisquePluie("Paris")).thenReturn(false);

        // Act
        String conseil = conseiller.conseiller("Paris");

        // Assert
        assertEquals("Manteau chaud obligatoire", conseil);
        // Pas de verify() car on teste l'ÉTAT (retour de la méthode)
        // et non le COMPORTEMENT (interactions avec le mock)
    }
}
```

Corrigé — Ex 2.4 (amorce)

```
@Test
void inscrire_EmailAvecMajuscules_SauvegardeEnMinuscules() {
    // Arrange
    ArgumentCaptor<User> userCaptor = ArgumentCaptor.forClass(User.class);
    when(repository.save(any(User.class)))
        .thenAnswer(inv -> inv.getArgument(0)); // retourne l'objet tel quel

    InscriptionRequest request = new InscriptionRequest(
        "Alice", "ALICE@EMAIL.COM", "motdepasse123"
    );

    // Act
    service.inscrire(request);

    // Assert
    verify(repository).save(userCaptor.capture());
    User userSauvegarde = userCaptor.getValue();
    assertEquals("alice@email.com", userSauvegarde.getEmail(),
        "L'email doit être converti en minuscules avant la sauvegarde");
}
```


Corrigé — Ex 3.1 (amorce InOrder)

```
@Test
void valider_ProcessusCompleet_EtapesEnOrdre() {
    // Arrange
    when(authService.authentifier(anyString())).thenReturn(utilisateurValide);
    when(tarifService.calculerPrix(any())).thenReturn(150.0);
    when(paieementService.debiter(anyString(), anyDouble()))
        .thenReturn("TX-001");
    when(factureService.generer(any())).thenReturn(new Facture());

    // Act
    processusService.valider(commande, "token-valid", "carte-001");

    // Assert – vérification de l'ordre
    InOrder ordre = inOrder(
        authService, validationService, stockService,
        tarifService, paieementService, factureService, notifService
    );
    ordre.verify(authService).authentifier("token-valid");
    ordre.verify(validationService).valider(commande);
    ordre.verify(stockService).verifier(commande);
    ordre.verify(tarifService).calculerPrix(commande);
    ordre.verify(paieementService).debiter("carte-001", 150.0);
    ordre.verify(stockService).mettreAJour(commande);
    ordre.verify(factureService).generer(commande);
    ordre.verify(notifService).confirmer(anyString(), any(Facture.class));
}
```

Exercices Jour 2 — Formation TDD Java · © 2026